

WarpBuild iOS CI 빌드 및 화면 캡처

React Native 앱을 macOS runner에서 빌드한 뒤
iOS Simulator에서 실행하고
화면과 진단 증거를 남기는 표준 절차

검증 완료

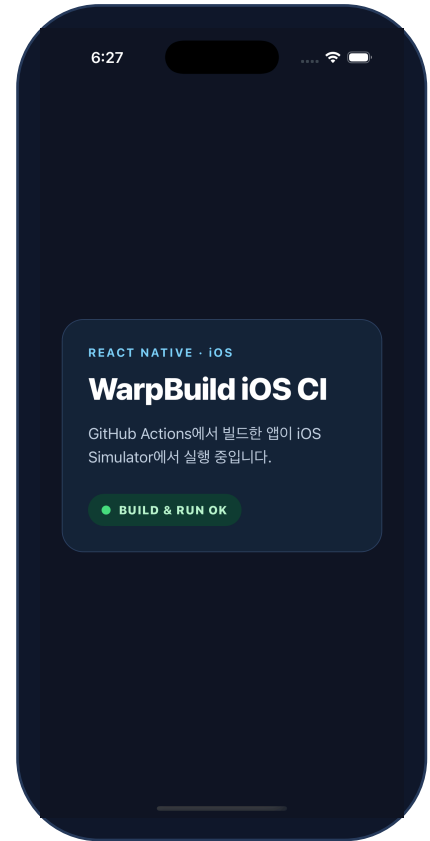
run [33364094460](#)에서 빌드, 앱 실행, PNG 캡처, artifact 업로드가 모두
성공했습니다.

대상 저장소

kentlee86/ios-build-test

워크플로

.github/workflows/ios-warpbuild.yml



실제 iPhone 16 Pro Simulator 캡처

RUNNER LABEL

**warp-macos-15-
arm64-6x**

할당된 runner 이름은 매 실행마다 달라질
수 있음

ELAPSED

2분 32초

성공 job 시작부터 완료까지

EVIDENCE

3개 파일

PNG, Simulator 로그, 빌드 환경

QUICK OVERVIEW

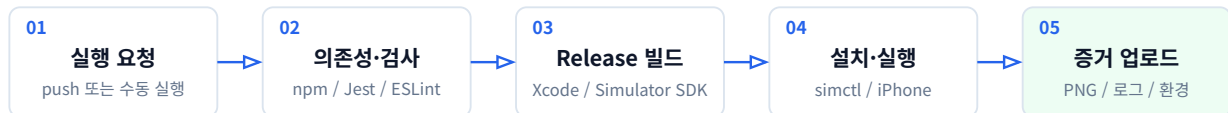
1 목적과 전체 흐름

이 문서가 증명하는 범위

서명 없는 Release 앱이 WarpBuild macOS runner에서 빌드되고 iOS Simulator에 설치·실행되어 화면이 렌더링됐음을 확인합니다. 실제 iPhone 기기, 코드 서명, TestFlight 또는 App Store 배포를 증명하는 절차는 아닙니다.

처리 흐름

GitHub Actions가 아래 순서로 한 번의 페루프 검증을 수행합니다.



시작 조건

- main 브랜치에 push
- Actions 화면에서 workflow_dispatch
- GitHub CLI의 gh workflow run

완료 조건

- job 결론이 success
- 앱 실행 결과에 bundle ID와 PID 표시
- ios-simulator.png가 비어 있지 않음
- ios-warpbuilt-evidence artifact 생성

검증된 기준값

항목	확인값
Git commit	f3f5b5d12d19b17fe6f033bb017d4d7f77d45db8
Runner	warp-6x-arm64-wu7rgcvyncf26ppr / group default
빌드 도구	Xcode 16.4, Node 22.22.0, Ruby 3.3.12, CocoaPods 1.15.2
Simulator	iPhone 16 Pro
앱 실행	com.kentlee.iosbuildtest: 6110

GitHub 웹 화면에서 실행

Actions → **iOS WarpBuild** → **Run workflow** → branch main → **Run workflow** 순서로 실행합니다. 완료 후 run 하단의 **Artifacts**에서 ios-warpbuilt-evidence를 내려받습니다.

PREREQUISITES

2 최초 1회 설정

2.1 계정과 저장소 준비

다음 네 항목을 먼저 확인합니다. UI 명칭은 서비스 개편에 따라 달라질 수 있으므로 상태를 기준으로 판단합니다.

1

WarpBuild와 GitHub 연결

WarpBuild에서 GitHub 조직에 접근할 계정으로 로그인합니다.

2

GitHub App 권한

kentlee86/ios-build-test가 설치 대상에 포함되어 있어야 합니다.

3

결제 상태

billing이 활성화되고 runner를 실행할 수 있는 credit 또는 balance가 있어야 합니다.

4

Stock runner set

macOS stock runner가 보이고 workflow label을 사용할 수 있어야 합니다.

2.2 저장소의 핵심 설정

현재 프로젝트에서는 아래 값을 기준으로 합니다. 다른 앱에 재사용할 때 bundle ID, workspace, scheme만 해당 프로젝트 값으로 바꿉니다.

설정	값과 의미
runs-on	warp-macos-15-arm64-6x - WarpBuild macOS 15 ARM64 runner label
APP_BUNDLE_ID	com.kentlee.iosbuildtest - Xcode target의 bundle ID와 일치
DERIVED_DATA	build/DerivedData - 빌드 산출물 위치를 고정
Xcode workspace	ios/IosBuildTest.xcworkspace
Xcode scheme	IosBuildTest
권한	contents: read - checkout에 필요한 최소 권한
시간 제한	timeout-minutes: 30

Workflow 파일이 기준

실행 정의의 SSOT는 .github/workflows/ios-warpbuild.yml입니다. 본문은 동작을 설명하고, 실행 명령과 핵심 YAML 발체는 뒤쪽 **부록 A**에 모았습니다.

Apple 인증서는 필요하지 않음

이 절차는 iphonesimulator용 서명 없는 앱을 CODE_SIGNING_ALLOWED=NO로 빌드합니다. 실제 기기 설치나 배포용 archive에는 별도의 인증서와 provisioning 설정이 필요합니다.

WORKFLOW DETAILS

3

빌드와 화면 캡처 동작

1

의존성과 정적 검사

npm ci 후 Jest와 ESLint를 실행합니다. Ruby 3.3과 Bundler를 설정하고 CocoaPods 의존성을 설치합니다.

2

Release Simulator 빌드

workspace와 scheme을 지정하고 iphonesimulator SDK로 서명 없이 빌드합니다.

3

Simulator 부팅과 앱 실행

사용 가능한 첫 iPhone을 선택해 부팅하고 .app을 설치한 뒤 bundle ID로 실행합니다.

4

증거 수집

10초 대기 후 PNG를 저장하고 최근 3분의 앱 로그와 도구 버전을 함께 업로드합니다.

Workflow 단계와 책임

단계 이름	수행 내용
Run JavaScript checks	Jest와 ESLint를 실행해 빌드 전 정적 검사를 마칩니다.
Build unsigned Release app for Simulator	Release 설정과 Simulator SDK로 서명 없이 앱을 빌드합니다.
Select and boot an iPhone Simulator	사용 가능한 iPhone UDID를 선택하고 부팅 완료까지 기다립니다.
Install, launch, and capture the app	앱 설치와 실행 후 PNG가 생성됐는지 확인합니다.
Upload iOS evidence	화면, 로그, 환경 metadata를 하나의 artifact로 보존합니다.

Artifact 구성

파일	판정에 쓰는 내용	보존
ios-simulator.png	앱이 실제로 표시된 Simulator 화면	14일
simulator.log	실행 시점 최근 3분의 IosBuildTest 프로세스 로그	14일
build-metadata.txt	Xcode, Node, Ruby, CocoaPods, Simulator 정보	14일

캡처 시점 조정

앱 초기화가 10초보다 오래 걸리면 sleep 10을 늘리거나, 화면 준비를 판정할 수 있는 신호로 대체합니다. 고정 대기 시간을 줄일 때는 빈 화면이나 launch screen이 캡처되지 않는지 다시 확인합니다.

RUN AND VERIFY

4 실행하고 결과 확인하기

4.1 실행 순서

저장소 루트에서 인증, 실행, 추적, 증거 다운로드 순으로 진행합니다. 복사 가능한 PowerShell 명령은 뒤쪽 **부록 A**에 있습니다.

1

GitHub 인증 확인

gh auth status로 현재 계정과 repository 접근 권한을 확인합니다.

2

Workflow 실행

main 브랜치에서 iOS WarpBuild를 수동 실행합니다.

3

Run 완료 추적

새 run ID를 얻고 job이 끝날 때까지 상태와 종료 코드를 확인합니다.

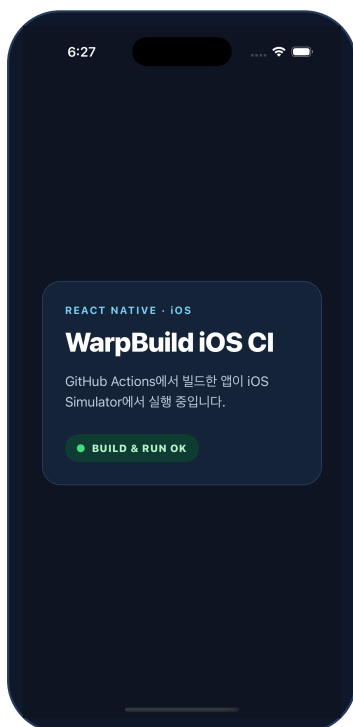
4

Artifact 회수

ios-warpbuild-evidence를 내려받아 PNG부터 직접 엽니다.

명령과 파일은 뒤쪽에 배치

복사해 실행할 PowerShell 명령과 Workflow 핵심 YAML은 **부록 A**에 있으며, 전체 원본은 `.github/workflows/ios-warpbuild.yml`을 참조합니다.



run 33364094460의 실제 캡처

4.2 성공 판정 체크리스트

- ✓ **Workflow** Build, run, and capture iOS job이 녹색 success
- ✓ **Xcode** 로그 끝에 `** BUILD SUCCEEDED **`
- ✓ **앱 실행** `com.kentlee.iosbuildtest: <PID>`가 출력됨
- ✓ **화면** WarpBuild iOS CI와 BUILD & RUN OK가 PNG에 표시됨
- ✓ **Artifact** PNG, 로그, metadata 세 파일이 모두 존재

화면을 직접 판정

job 성공만으로 UI가 올바르게 보였다고 단정하지 않습니다. PNG를 열어 문구, 배치, 잘림, 빈 화면 여부를 확인합니다.

ONLY REQUIRED WHEN ONBOARDING IS BLOCKED

5

FVE_008일 때 지원 요청

오류 코드 정정

지원팀에 보낸 핵심 오류는 FVE_008입니다. Runner 할당 단계에서 함께 보인 코드는 RNR_ALLOC_007입니다. FVE_007로 기록하지 않습니다.

언제 메일을 보내는가

billing과 GitHub App 연결이 정상인데 stock runner가 보이지 않고, runner 생성 또는 onboarding 요청이 FVE_008로 거절될 때 한 번 보냅니다. Runner가 이미 job에 할당된다면 이 단계는 건너뛰니다.

메일에 넣을 증거

- GitHub organization과 repository
- GitHub App installation ID
- billing 활성 및 잔액 상태
- stock runner 목록이 비어 있다는 상태
- RNR_ALLOC_007과 FVE_008 원문
- 영향을 받은 Actions run URL
- runner 미배정이라 step log가 없다는 설명

요청할 조치

- Cloud Runner onboarding suspension 해제
- stock runner set 복구
- 조직이 다시 runner를 할당할 수 있는지 확인
- 조치 완료 회신 요청

지원 메일 템플릿

To: support@warpbuild.com
Subject: FVE_008 - Please unsuspend Cloud Runner onboarding for <organization>

Hello WarpBuild Support,

Please unsuspend Cloud Runner onboarding for our GitHub organization and restore the stock runner set.

- Organization: <organization>
- Repository: <owner/repository>
- GitHub App installation ID: <installation-id>
- Billing status: active, with available balance
- Runner list: empty / stock runners unavailable
- Allocation error: RNR_ALLOC_007
- Runner creation error: FVE_008
- Affected Actions runs: <run-url-1>, <run-url-2>

No runner is assigned, so there are no job step logs. Please let us know when the organization is unblocked and the stock runner set is restored.

Thanks,
<name>

회신을 받은 뒤

1

추가 설정 없이 재실행

동일한 workflow를 main에서 수동 실행합니다.

2

Runner 배정 확인

Set up runner가 수 초 안에 시작되는지 봅니다.

3

페루프 확인

빌드 성공뿐 아니라 다운로드한 PNG까지 직접 확인합니다.

운영 기준

지원 메일은 runner onboarding이 막힌 경우에만 필요합니다. 정상 운영에서는 workflow 실행, run 추적, artifact 다운로드의 세 단계만 반복하면 됩니다.

REFERENCE

A PowerShell 명령 참조

본문과 분리된 실행 코드

아래 명령은 Windows PowerShell에서 그대로 복사해 사용할 수 있습니다. 저장소 이름이나 workflow 이름을 바꾸면 --repo와 --workflow 값도 함께 바꿉니다.

A.1 Workflow 실행과 완료 추적

```
gh auth status

gh workflow run "iOS WarpBuild" `
  --repo kentlee86/ios-build-test `
  --ref main

Start-Sleep -Seconds 3
$runId = gh run list `
  --repo kentlee86/ios-build-test `
  --workflow "iOS WarpBuild" `
  --limit 1 --json databaseId `
  --jq "[0].databaseId"

gh run watch $runId `
  --repo kentlee86/ios-build-test `
  --exit-status --interval 10
```

A.2 Artifact 다운로드와 화면 열기

```
$evidence = Join-Path $PWD "artifacts\$runId"
New-Item -ItemType Directory -Force -Path $evidence | Out-Null

gh run download $runId `
  --repo kentlee86/ios-build-test `
  --name ios-warpbuild-evidence `
  --dir $evidence

Invoke-Item (Join-Path $evidence "ios-simulator.png")
```

종료 코드도 확인

gh run watch --exit-status는 workflow 실패 시 0이 아닌 종료 코드를 반환합니다. 명령이 끝났다는 사실과 workflow가 성공했다는 사실을 구분합니다.

SOURCE REFERENCE

B Workflow 파일 참조

전체 원본이 SSOT

아래 내용은 핵심 발췌입니다. 실제 실행 기준은 repository의 `.github/workflows/ios-warpbuild.yml`이며, public 원본은 [GitHub에서 열기](#)로 확인합니다.

B.1 Trigger, runner, 환경값

```
name: iOS WarpBuild

on:
  push:
    branches: [main]
    workflow_dispatch:

permissions:
  contents: read

jobs:
  build-run-screenshot:
    name: Build, run, and capture iOS
    runs-on: warp-macos-15-arm64-6x
    timeout-minutes: 30
    env:
      APP_BUNDLE_ID: com.kentlee.iosbuildtest
      DERIVED_DATA: build/DerivedData
```

B.2 Release Simulator 빌드

```
- name: Build unsigned Release app for Simulator
  run: |
    xcodebuild \
      -workspace ios/IosBuildTest.xcworkspace \
      -scheme IosBuildTest \
      -configuration Release \
      -sdk iphonesimulator \
      -destination 'generic/platform=iOS Simulator' \
      -derivedDataPath "$DERIVED_DATA" \
      CODE_SIGNING_ALLOWED=NO build
```

B.3 설치·실행·캡처

```
- name: Install, launch, and capture the app
  shell: bash
  run: |
    APP_PATH="$DERIVED_DATA/Build/Products/\
    Release-iphonesimulator/IosBuildTest.app"
    xcrun simctl install "$SIMULATOR_UDID" "$APP_PATH"
    xcrun simctl launch "$SIMULATOR_UDID" "$APP_BUNDLE_ID"
    sleep 10
    xcrun simctl io "$SIMULATOR_UDID" screenshot \
      artifacts/ios-simulator.png
    test -s artifacts/ios-simulator.png
```

B.4 증거 업로드

```
- name: Upload iOS evidence
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: ios-warpbuild-evidence
    path: artifacts/
    if-no-files-found: warn
    retention-days: 14
```